

InfluxDB 3 Enterprise — Self-Hosted Deployment Guide

Scope: This guide covers self-hosted (bare-metal / VM) deployments of InfluxDB 3 Enterprise on Linux. Docker and Kubernetes deployments are out of scope here and will be covered separately.

Reference version: InfluxDB 3 Enterprise v3.9.x

Source: Adapted from the official InfluxData documentation at <https://docs.influxdata.com/influxdb3/enterprise/>

Table of Contents

1. [Overview and Architecture](#)
 2. [System Requirements](#)
 3. [Installation](#)
 4. [Initial Setup and First Start](#)
 5. [License Activation](#)
 6. [Multi-Node Cluster Setup](#)
 7. [Token Management](#)
 8. [Database and Table Management](#)
 9. [Writing Data](#)
 10. [Querying Data](#)
 11. [Migrating from InfluxDB v1 / v2](#)
 12. [Caches: LVC and DVC](#)
 13. [File Indexes](#)
 14. [Object Storage Configuration](#)
 15. [Processing Engine and Python Plugins](#)
 16. [System Tables and Monitoring](#)
 17. [Performance Tuning](#)
 18. [Performance Upgrade Preview \(PachaTree\)](#)
 19. [Security Hardening](#)
 20. [Backup and Restore](#)
 21. [Upgrading InfluxDB 3 Enterprise](#)
 22. [Troubleshooting](#)
-

1. Overview and Architecture

InfluxDB 3 Enterprise is a time-series database built on the InfluxDB 3 Core open source release. It uses a **diskless architecture** that persists data as Parquet files in object storage and scales horizontally.

1.1 Key features inherited from Core

- Diskless architecture with object storage support (or local disk)
- Sub-10 ms last-value queries; ~30 ms distinct metadata queries
- Embedded Python virtual machine for plugins and triggers
- Parquet file persistence
- Compatibility with InfluxDB 1.x and 2.x write APIs

1.2 Enterprise-only additions

- Historical query capability with single-series indexing
- High availability across multiple nodes
- Read replicas
- Row-level delete support (coming soon)
- Integrated admin UI (coming soon)

1.3 Data model

InfluxDB 3 term	Equivalent in v1	Equivalent in v2
Database	db/retention_policy	bucket
Table	measurement	measurement
Tag column	tag	tag
Field column	field	field

Each table has a primary key — the ordered set of tag columns plus `time`. The primary key is set on first write and is **immutable**. Use tags for unique identifying dimensions (`sensor_id`, `host`, `region`); use fields for measured values.

1.4 Node modes

A single InfluxDB 3 Enterprise binary can run in different **modes** depending on the role you want a node to play:

Mode	Purpose
<code>ingest</code>	Accepts writes, builds the WAL, snapshots data
<code>query</code>	Serves queries; reads Parquet from object storage
<code>compact</code>	Runs background compaction of Gen0/Gen1 files
<code>process</code>	Runs Processing Engine triggers and plugins
<code>all</code>	Combined mode (default for single-node installs)

2. System Requirements

2.1 Operating systems supported

- Linux (x86_64 / AMD64 and ARM64 / AArch64)
- macOS (Apple Silicon / ARM64)
- Windows (AMD64)

For self-hosted production, **Linux DEB or RPM** is the recommended path.

2.2 Hardware sizing (general guidance)

Role	CPU	RAM	Disk (local)
Single-node lab	4 cores	8 GB	50 GB SSD
Small production ingest	8 cores	16-32 GB	100 GB SSD
Production query	8-16 cores	32-64 GB	100 GB SSD (cache)
Production compactor	8 cores	16-32 GB	100 GB SSD

Object storage capacity should be planned based on retention period × ingest rate × compression ratio (typically 5-20×).

2.3 Object storage options

- **Local filesystem** — fastest, but limited to one host; suitable for dev/lab.
- **Amazon S3** — natively supported.
- **Azure Blob Storage** — natively supported.
- **Google Cloud Storage** — natively supported.
- **S3-compatible** — MinIO, Ceph (Ceph requires `--aws-s3-custom-backend ceph`).

2.4 Network requirements

- Default HTTP API port: **8181** (configurable with `--http-bind`)
 - Outbound access to object storage endpoints
 - For multi-node: every node needs read/write access to the same object store
-

3. Installation

InfluxDB 3 Enterprise on Linux can be installed in three ways. For production, prefer the DEB/RPM packages because they install a hardened systemd unit file.

3.1 Method 1 — DEB-based systems (Debian, Ubuntu)

```
bash
```

```
# Download the InfluxData GPG key
curl --silent --location -O \
  https://repos.influxdata.com/influxdata-archive.key

# Verify the key fingerprint
gpg --show-keys --with-fingerprint --with-colons ./influxdata-archive.key 2>&1 \
  | grep -q '^fpr:\+24C975CBA61A024EE1B631787C3D57159FC2F927:$' \
  && cat influxdata-archive.key \
  | gpg --dearmor \
  | sudo tee /usr/share/keyrings/influxdata-archive.gpg > /dev/null

# Add the InfluxData APT repository
echo 'deb [signed-by=/usr/share/keyrings/influxdata-archive.gpg] https://repos.influxdata.com/debian stable main' \
  | sudo tee /etc/apt/sources.list.d/influxdata.list

# Install
sudo apt-get update
sudo apt-get install influxdb3-enterprise
```

3.2 Method 2 — RPM-based systems (Rocky Linux, RHEL, Amazon Linux, CentOS)

```
bash
```

```
# Download the GPG key
curl --silent --location -O \
  https://repos.influxdata.com/influxdata-archive.key

sudo mkdir -p /usr/share/influxdata-archive-keyring/keyrings

# Verify and install the key
gpg --show-keys --with-fingerprint --with-colons ./influxdata-archive.key 2>&1 \
  | grep -q '^fpr:\+24C975CBA61A024EE1B631787C3D57159FC2F927:$' \
  && sudo cp ./influxdata-archive.key \
    /usr/share/influxdata-archive-keyring/keyrings/influxdata-archive.asc

# Configure the YUM repository
cat <<EOF | sudo tee /etc/yum.repos.d/influxdata.repo
[influxdata]
name = InfluxData Repository - Stable
baseurl = https://repos.influxdata.com/stable/\$basearch/main
enabled = 1
gpgcheck = 1
gpgkey = file:///usr/share/influxdata-archive-keyring/keyrings/influxdata-archive.asc
EOF

# Install
sudo yum install influxdb3-enterprise
```

3.3 Method 3 — Quick installer script (lab use)

```
bash

curl -O https://www.influxdata.com/d/install_influxdb3.sh \
  && sh install_influxdb3.sh enterprise
```

Best for evaluation and lab environments. The script always pulls the latest release.

3.4 Method 4 — Direct binary download

For air-gapped environments or custom installs, download the tarball:

```
bash
```

```
# Linux AMD64
curl -LO https://dl.influxdata.com/influxdb/releases/influxdb3-enterprise-3.9.1_linux_amd64.tar.gz

# Verify checksum
curl -LO https://dl.influxdata.com/influxdb/releases/influxdb3-enterprise-3.9.1_linux_amd64.tar.gz.sha256sum
sha256sum -c influxdb3-enterprise-3.9.1_linux_amd64.tar.gz.sha256

# Extract
tar -xzf influxdb3-enterprise-3.9.1_linux_amd64.tar.gz
```

Important: The `influxdb3` binary requires the adjacent `python/` directory to function. Keep them together in the same parent directory and add the parent directory to `PATH`. Do **not** move the binary out of its install directory.

```
bash

# Recommended layout
/opt/influxdb3/
├─ influxdb3
└─ python/

export PATH=/opt/influxdb3:$PATH
```

3.5 Verify installation

```
bash

influxdb3 --version
```

Expected output:

```
influxdb3 3.9.1 (Enterprise)
```

4. Initial Setup and First Start

4.1 Default file locations (DEB / RPM install)

Path	Purpose
<code>/usr/bin/influxdb3</code>	Binary

Path	Purpose
<code>/etc/influxdb3/influxdb3-enterprise.conf</code>	TOML configuration
<code>/var/lib/influxdb3/data</code>	Local data directory
<code>/var/lib/influxdb3/plugins</code>	Processing Engine plugin directory
<code>/var/log/influxdb3/</code>	Logs (when not using journald)
<code>/lib/systemd/system/influxdb3-enterprise.service</code>	systemd unit file

4.2 Default TOML configuration values

The packaged `/etc/influxdb3/influxdb3-enterprise.conf` ships with:

Setting	Default
<code>object-store</code>	<code>file</code>
<code>data-dir</code>	<code>/var/lib/influxdb3/data</code>
<code>plugin-dir</code>	<code>/var/lib/influxdb3/plugins</code>
<code>node-id</code>	<code>primary-node</code>
<code>cluster-id</code>	<code>primary-cluster</code>
<code>mode</code>	<code>all</code>

4.3 Edit the configuration

For a single-node install backed by local disk, the defaults are usually sufficient. Edit `/etc/influxdb3/influxdb3-enterprise.conf` to change values such as bind address, HTTP port, or object store.

Example excerpt for a node bound to a non-default port:

```
toml
```



```
node-id           = "ingest-node-01"
cluster-id        = "datapundit-cluster"
mode              = "all"
object-store      = "file"
data-dir          = "/var/lib/influxdb3/data"
plugin-dir        = "/var/lib/influxdb3/plugins"
http-bind         = "0.0.0.0:8181"
log-filter        = "info"
```

4.4 Start the service (systemd)

The unit is **enabled but not started** after install so you can configure first.

```
bash

# Start the service
sudo systemctl start influxdb3-enterprise

# Confirm it's running
sudo systemctl status influxdb3-enterprise

# Tail logs
sudo journalctl -u influxdb3-enterprise -f
```

4.5 Start the service (SysV init systems)

```
bash

# Enable
sudo sed -i 's/ENABLED=no/ENABLED=yes/' /etc/default/influxdb3-enterprise

# Start
sudo /etc/init.d/influxdb3-enterprise start

# Status
sudo /etc/init.d/influxdb3-enterprise status

# Logs
sudo tail -f /var/lib/influxdb3/influxdb3-enterprise.log
```

4.6 Run manually (binary install only — for debugging)

```
bash
```

```
influxdb3 serve \  
  --node-id ingest-node-01 \  
  --cluster-id datapundit-cluster \  
  --mode all \  
  --object-store file \  
  --data-dir /var/lib/influxdb3/data \  
  --plugin-dir /var/lib/influxdb3/plugins
```

4.7 Health check

```
bash  
  
curl -s http://localhost:8181/health  
curl -s http://localhost:8181/ping
```

A 200 response confirms the server is up.

5. License Activation

InfluxDB 3 Enterprise **requires an active license to start**. There are three license types:

Type	Use case
home	Free for non-commercial home use
trial	Free 30-day evaluation
commercial	Paid production license

5.1 Interactive activation (default)

On first start, the server prints an onboarding URL and prompts in the terminal. Follow the instructions to:

1. Provide an email address.
2. Choose a license type.
3. Complete email verification.
4. The license file is automatically downloaded and stored.

5.2 Non-interactive activation

For scripted or headless installs, pass `--license-type` to `influxdb3 serve`:

```
bash
influxdb3 serve \
  --node-id ingest-node-01 \
  --cluster-id datapundit-cluster \
  --object-store file \
  --data-dir /var/lib/influxdb3/data \
  --license-type trial \
  --license-email shri@example.com
```

5.3 View the active license

```
bash
influxdb3 show license --token $ADMIN_TOKEN
```

Output includes license type, email, expiration date, and resource limits.

5.4 Renewing a license

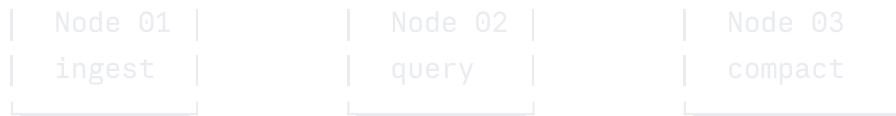
Replace the license file under the data directory with the new one issued by InfluxData and restart the service.

6. Multi-Node Cluster Setup

A multi-node Enterprise cluster shares state through a **single object store** and a **cluster ID**. Every node points to the same `--cluster-id` and the same object store; nodes are differentiated by `--node-id` and `--mode`.

6.1 Architecture (three-node example)





6.2 Pre-deployment checklist

- ☐ Same `cluster-id` on every node.
- ☐ Unique `node-id` on every node.
- ☐ Identical object-store configuration (bucket, region, credentials).
- ☐ License activated on every node.
- ☐ Same InfluxDB 3 Enterprise version on every node.
- ☐ Network connectivity and firewall rules between nodes and object store.
- ☐ If using Processing Engine: identical plugin files at `--plugin-dir` on every node that runs them.

6.3 Configure each node

Node 01 — ingest

```
bash

influxdb3 serve \
  --node-id ingest-01 \
  --cluster-id datapundit-cluster \
  --mode ingest \
  --object-store s3 \
  --bucket influxdb3-data \
  --aws-access-key-id $AWS_ACCESS_KEY_ID \
  --aws-secret-access-key $AWS_SECRET_ACCESS_KEY \
  --aws-default-region ap-south-1
```

Node 02 — query

```
bash

influxdb3 serve \
  --node-id query-01 \
  --cluster-id datapundit-cluster \
  --mode query \
  --object-store s3 \
  --bucket influxdb3-data \
  --aws-default-region ap-south-1
```

Node 03 — compact

```
bash

influxdb3 serve \
  --node-id compact-01 \
  --cluster-id datapundit-cluster \
  --mode compact \
  --object-store s3 \
  --bucket influxdb3-data \
  --aws-default-region ap-south-1
```

6.4 Verify cluster nodes

From any node:

```
bash

influxdb3 show nodes --token $ADMIN_TOKEN
```

You should see all configured nodes listed with their mode and last-seen timestamp.

6.5 Routing client traffic

Workload	Send to
Writes (Telegraf, client libraries)	Ingest nodes
SQL/InfluxQL queries (Grafana, dashboards)	Query nodes
HTTP-trigger Processing Engine plugins	Process-mode nodes

Use a load balancer (HAProxy, NGINX, AWS ALB) in front of nodes serving the same role.

7. Token Management

InfluxDB 3 Enterprise uses **tokens** for authentication. There are two categories:

- **Admin tokens** — full control of the server (databases, tables, tokens, plugins, license).
- **Resource tokens** — scoped to specific databases with `read`, `write`, or `create` permissions.

The first token created is the **operator token** (`_admin`) and cannot be deleted.

7.1 Create the operator admin token (first run)

If the server starts without authorization, it prints the operator token once. Save it immediately.

```
bash
influxdb3 create token --admin
```

Sample output:

```
New token created successfully!
Token: apiv3_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Save this token securely – it will not be shown again.
```

Export it for use by the CLI:

```
bash
export INFLUXDB3_AUTH_TOKEN=apiv3_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

7.2 Create a named admin token

```
bash
influxdb3 create token \
  --admin \
  --name shri-admin \
  --expiry 90d \
  --token $INFLUXDB3_AUTH_TOKEN
```

7.3 Create a resource token

```
bash
```

```
# Read + write on database "metrics"
influxdb3 create token \
  --name telegraf-writer \
  --database metrics \
  --permission read,write \
  --token $INFLUXDB3_AUTH_TOKEN

# Read-only on multiple databases
influxdb3 create token \
  --name grafana-reader \
  --database metrics,iot \
  --permission read \
  --token $INFLUXDB3_AUTH_TOKEN

# Permission to create new databases
influxdb3 create token \
  --name app-bootstrapper \
  --permission create \
  --token $INFLUXDB3_AUTH_TOKEN
```

7.4 List tokens

```
bash

influxdb3 show tokens --token $INFLUXDB3_AUTH_TOKEN
```

7.5 Revoke a token

```
bash

influxdb3 delete token \
  --name telegraf-writer \
  --token $INFLUXDB3_AUTH_TOKEN
```

7.6 Recover a lost admin token

If you lose all admin tokens, start the server with the recovery server enabled:

```
bash
```

```
influxdb3 serve \  
  --node-id ingest-01 \  
  --cluster-id datapundit-cluster \  
  --object-store file \  
  --data-dir /var/lib/influxdb3/data \  
  --admin-token-recovery-http-bind 127.0.0.1:8182
```

Then call the recovery endpoint:

```
bash  
  
curl -X POST http://127.0.0.1:8182/api/v3/configure/admin_token/regenerate
```

The recovery server shuts down automatically after the new token is issued.

7.7 Disable authentication (lab only)

```
bash  
  
influxdb3 serve --without-auth ...
```

Warning: Never use `--without-auth` in production.

8. Database and Table Management

8.1 Create a database

```
bash  
  
influxdb3 create database metrics --token $INFLUXDB3_AUTH_TOKEN
```

With a retention period:

```
bash  
  
influxdb3 create database metrics \  
  --retention-period 30d \  
  --token $INFLUXDB3_AUTH_TOKEN
```

Naming rule: Database names cannot contain underscores when created via the API.

8.2 List databases

```
bash
```

```
influxdb3 show databases --token $INFLUXDB3_AUTH_TOKEN
```

8.3 Update database retention

```
bash
```

```
influxdb3 update database metrics \  
  --retention-period 90d \  
  --token $INFLUXDB3_AUTH_TOKEN
```

8.4 Delete a database

Soft delete (default — retains catalog entry, schedules data for deletion):

```
bash
```

```
influxdb3 delete database metrics --token $INFLUXDB3_AUTH_TOKEN
```

Hard delete (removes everything immediately):

```
bash
```

```
influxdb3 delete database metrics \  
  --hard-delete now \  
  --yes \  
  --token $INFLUXDB3_AUTH_TOKEN
```

8.5 Create a table explicitly

```
bash
```

```
influxdb3 create table cpu_usage \  
  --database metrics \  
  --tags host,region \  
  --fields usage_user:float64,usage_sys:float64 \  
  --token $INFLUXDB3_AUTH_TOKEN
```

A table is also created automatically on first write (schema-on-write). Tag column order on first write becomes the immutable primary key.

8.6 Set a per-table retention period

```
bash

influxdb3 update table cpu_usage \
  --database metrics \
  --retention-period 7d \
  --token $INFLUXDB3_AUTH_TOKEN
```

8.7 View retention policies

```
bash

influxdb3 show retention --database metrics --token $INFLUXDB3_AUTH_TOKEN
```

8.8 Delete a table

```
bash

influxdb3 delete table cpu_usage \
  --database metrics \
  --token $INFLUXDB3_AUTH_TOKEN
```

9. Writing Data

9.1 Choose the right write endpoint

Workload	Endpoint
New v3-native workloads	POST /api/v3/write_lp
Existing v2 clients (Telegraf v2 plugin, v2 SDKs)	POST /api/v2/write
Existing v1 clients	POST /write

9.2 Line protocol primer

```
measurement[,tag_key=tag_value...] field_key=field_value[,...] [timestamp]
```

Example:

```
cpu,host=srv01,region=ap-south-1 usage_user=55.2,usage_sys=12.1
17144640000000000000
```

9.3 Write via the v3 HTTP API

bash

```
curl -X POST "http://localhost:8181/api/v3/write_lp?db=metrics&precision=nanosecond" \
  -H "Authorization: Bearer $INFLUXDB3_AUTH_TOKEN" \
  -H "Content-Type: text/plain" \
  --data-binary 'cpu,host=srv01 usage_user=55.2,usage_sys=12.1'
```

9.4 Write via the CLI

bash

```
influxdb3 write \
  --database metrics \
  --token $INFLUXDB3_AUTH_TOKEN \
  --precision ns \
  'cpu,host=srv01 usage_user=55.2,usage_sys=12.1'
```

From a file:

bash

```
influxdb3 write \
  --database metrics \
  --token $INFLUXDB3_AUTH_TOKEN \
  --file /tmp/lineprotocol.txt
```

9.5 Write via Telegraf (v2 output plugin)

In `telegraf.conf`:

toml

```
[[outputs.influxdb_v2]]
  urls      = ["http://ingest-01.example.com:8181"]
  token     = "apiv3_XXXXXXXXXXXXXXXXXXXX"
  organization = ""
  bucket    = "metrics"
```

Restart Telegraf:

```
bash
sudo systemctl restart telegraf
```

9.6 Timestamp precision matrix

Precision	v1 (/write)	v2 (/api/v2/write)	v3 (/api/v3/write_lp)
auto-detect	X	X	✓ (default)
second	s	s	second / s
millisecond	ms	ms	millisecond / ms
microsecond	u, μ	us	microsecond / us
nanosecond	ns	ns	nanosecond / ns
minute	m	X	X
hour	h	X	X

All timestamps are converted to nanoseconds internally.

9.7 Schema design recommendations

- Put unique identifying dimensions in **tags** (sensor_id, host, customer_id).
- Put numeric measurements in **fields**.
- Avoid high-cardinality tags unless you understand the storage and query cost.
- Plan tag order on the first write — it sets the immutable primary key.

10. Querying Data

InfluxDB 3 Enterprise supports two query languages: **SQL** (preferred) and **InfluxQL** (for v1 compatibility).

10.1 Query via the CLI (SQL)

```
bash
```

```
influxdb3 query \  
  --database metrics \  
  --token $INFLUXDB3_AUTH_TOKEN \  
  "SELECT host, mean(usage_user) AS avg_user  
  FROM cpu  
  WHERE time > now() - INTERVAL '1 hour'  
  GROUP BY host  
  ORDER BY avg_user DESC"
```

10.2 Query via the CLI (InfluxQL)

```
bash  
  
influxdb3 query \  
  --database metrics \  
  --language influxql \  
  --token $INFLUXDB3_AUTH_TOKEN \  
  "SELECT mean(usage_user) FROM cpu WHERE time > now() - 1h GROUP BY host"
```

10.3 Query via HTTP (SQL JSON)

```
bash  
  
curl -X POST "http://localhost:8181/api/v3/query_sql" \  
  -H "Authorization: Bearer $INFLUXDB3_AUTH_TOKEN" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "db": "metrics",  
    "q": "SELECT host, count(*) FROM cpu GROUP BY host",  
    "format": "json"  
  }'
```

Supported `format` values: `json`, `jsonl`, `csv`, `pretty`, `parquet`.

10.4 Query via Flight SQL

For high-throughput programmatic access, use the Flight SQL endpoint on port 8181 over gRPC. Client libraries are available in Python, Go, Java, JavaScript, C#, and others.

Python example:

```
python
```

```
from influxdb_client_3 import InfluxDBClient3

client = InfluxDBClient3(
    host="ingest-01.example.com",
    token="apiv3_XXXXXXXXXXXXXXXX",
    database="metrics"
)

result = client.query("SELECT * FROM cpu WHERE time > now() - INTERVAL '5 minutes'")
print(result.to_pandas())
```

10.5 Connect from Grafana

1. In Grafana, install the **FlightSQL** or **InfluxDB** datasource plugin.
2. Set URL to the **query node**: `http://query-01.example.com:8181`
3. Authentication: bearer token (your read-only resource token).
4. Database: `metrics`.

10.6 Useful CLI helpers

```
bash

# Show table list and schema
influxdb3 show tables --database metrics --token $INFLUXDB3_AUTH_TOKEN

# Show retention
influxdb3 show retention --database metrics --token $INFLUXDB3_AUTH_TOKEN

# Show plugins
influxdb3 show plugins --token $INFLUXDB3_AUTH_TOKEN

# Show summary (databases, tables, triggers)
influxdb3 show summary --database metrics --token $INFLUXDB3_AUTH_TOKEN
```

11. Migrating from InfluxDB v1 / v2

11.1 v1 client compatibility

Existing v1 clients work against the `/write` and `/query` endpoints with no code changes (apart from token-based auth):

```
bash
```

```
curl -X POST "http://localhost:8181/write?db=metrics&precision=ns" \  
  -H "Authorization: Token $INFLUXDB3_AUTH_TOKEN" \  
  --data-binary 'cpu,host=srv01 usage_user=55.2'
```

11.2 v2 client compatibility

v2 clients send writes to `/api/v2/write` with `org` and `bucket` parameters; in InfluxDB 3, `org` is ignored and `bucket` maps to the database name:

```
bash
```

```
curl -X POST "http://localhost:8181/api/v2/write?org=&bucket=metrics&precision=ns" \  
  -H "Authorization: Token $INFLUXDB3_AUTH_TOKEN" \  
  --data-binary 'cpu,host=srv01 usage_user=55.2'
```

11.3 Migrating historical data

There is no native bulk importer that reads from v1/v2 storage. Recommended approaches:

1. Export to line protocol from the source InfluxDB:

- v1: `influx_inspect export -datadir ... -waldir ... -out lp.txt`
- v2: `influx export all-instance --output-path ...`

2. Replay into InfluxDB 3:

```
bash
```

```
influxdb3 write --database metrics --file lp.txt --token $INFLUXDB3_AUTH_TOKEN
```

11.4 Continuous queries / tasks

There is no direct equivalent in v3. Reimplement these as **Processing Engine scheduled triggers** (see section 15).

12. Caches: LVC and DVC

12.1 Last Value Cache (LVC)

The LVC keeps the most recent N values for selected fields, accelerating "latest reading" queries.

Create an LVC

```
bash

influxdb3 create last_cache \
  --database metrics \
  --table cpu \
  --name cpu_lvc \
  --key-columns host \
  --value-columns usage_user,usage_sys \
  --count 5 \
  --token $INFLUXDB3_AUTH_TOKEN
```

Query the LVC

```
sql

SELECT * FROM last_cache('cpu', 'cpu_lvc');
```

Delete an LVC

```
bash

influxdb3 delete last_cache \
  --database metrics \
  --table cpu \
  --name cpu_lvc \
  --token $INFLUXDB3_AUTH_TOKEN
```

12.2 Distinct Value Cache (DVC)

The DVC caches distinct values of one or more columns, speeding up `SHOW TAG VALUES` and `tag_values()` queries.

Create a DVC

```
bash
```



```
influxdb3 create distinct_cache \  
  --database metrics \  
  --table cpu \  
  --name cpu_dvc \  
  --columns host,region \  
  --max-cardinality 10000 \  
  --max-age 1h \  
  --token $INFLUXDB3_AUTH_TOKEN
```

Query the DVC

```
sql  
  
SELECT * FROM distinct_cache('cpu', 'cpu_dvc');  
SELECT DISTINCT host FROM cpu;           -- automatically uses DVC if enabled
```

Delete a DVC

```
bash  
  
influxdb3 delete distinct_cache \  
  --database metrics \  
  --table cpu \  
  --name cpu_dvc \  
  --token $INFLUXDB3_AUTH_TOKEN
```

12.3 Where caches run

In a multi-node cluster, LVC and DVC are populated and queried only on **query** nodes. Ingest nodes do not maintain these caches, which keeps ingest write paths lean.

13. File Indexes

A file index speeds up single-series queries by indexing one or more columns across persisted Parquet files.

13.1 Create a file index

```
bash
```

```
influxdb3 create file_index \  
  --database metrics \  
  --table cpu \  
  --columns host,region \  
  --token $INFLUXDB3_AUTH_TOKEN
```

13.2 List file indexes

```
bash
```

```
influxdb3 show file_indexes --database metrics --token $INFLUXDB3_AUTH_TOKEN
```

13.3 Delete a file index

```
bash
```

```
influxdb3 delete file_index \  
  --database metrics \  
  --table cpu \  
  --token $INFLUXDB3_AUTH_TOKEN
```

13.4 When to use a file index

Use file indexes when:

- You have high-cardinality tag values.
 - Most queries filter by a small subset of those tag values.
 - You can tolerate a small write-path overhead in exchange for much faster reads.
-

14. Object Storage Configuration

14.1 Local filesystem (single-node only)

```
bash
```

```
--object-store file \  
--data-dir /var/lib/influxdb3/data
```

14.2 Amazon S3

```
bash
```

```
--object-store s3 \  
--bucket influxdb3-data \  
--aws-access-key-id $AWS_ACCESS_KEY_ID \  
--aws-secret-access-key $AWS_SECRET_ACCESS_KEY \  
--aws-default-region ap-south-1
```

For ephemeral (rotating) credentials, point InfluxDB at a credentials file that gets refreshed on disk:

```
bash  
  
--aws-credentials-file /var/secrets/aws/credentials
```

14.3 S3-compatible (MinIO)

```
bash  
  
--object-store s3 \  
--bucket influxdb3-data \  
--aws-endpoint http://minio.internal:9000 \  
--aws-access-key-id minioadmin \  
--aws-secret-access-key minioadmin \  
--aws-default-region us-east-1 \  
--aws-allow-http
```

14.4 S3-compatible (Ceph)

```
bash  
  
--object-store s3 \  
--aws-s3-custom-backend ceph \  
--bucket influxdb3-data \  
--aws-endpoint https://ceph.internal:8080 \  
--aws-access-key-id $CEPH_ACCESS_KEY \  
--aws-secret-access-key $CEPH_SECRET_KEY
```

The `--aws-s3-custom-backend ceph` flag handles ETag quoting required for conditional PUTs on Ceph.

14.5 Azure Blob Storage

```
bash
```

```
--object-store azure \  
--bucket influxdb3-container \  
--azure-storage-account-name $AZURE_ACCOUNT \  
--azure-storage-access-key $AZURE_KEY \  
--azure-endpoint https://myaccount.blob.core.windows.net
```

14.6 Google Cloud Storage

```
bash  
  
--object-store google \  
--bucket influxdb3-data \  
--google-service-account /etc/influxdb3/gcp-sa.json
```

14.7 TLS for object storage

Skip TLS verification (for testing only):

```
bash  
  
--object-store-disable-tls-verify
```

Provide a custom root CA:

```
bash  
  
--object-store-root-ca /etc/ssl/certs/internal-ca.pem
```

15. Processing Engine and Python Plugins

15.1 What is the Processing Engine?

The Processing Engine is an embedded Python VM that runs inside InfluxDB 3 Enterprise. It executes Python plugin code in response to **triggers**:

Trigger type	Spec syntax	When it runs
Data write	<code>table:<name></code> or <code>all_tables</code>	When ingest flushes WAL data for the table
Scheduled	<code>every:<duration></code> or <code>cron:<expr></code>	At time intervals
HTTP request	<code>request:<path></code>	On HTTP call to <code>/api/v3/engine/<path></code>

15.2 Enable the Processing Engine

DEB/RPM installs ship with the engine enabled (plugin dir at `/var/lib/influxdb3/plugins`). For binary installs, pass `--plugin-dir`:

```
bash

influxdb3 serve \
  --node-id node01 \
  --cluster-id datapundit-cluster \
  --object-store file \
  --data-dir /var/lib/influxdb3/data \
  --plugin-dir /var/lib/influxdb3/plugins
```

15.3 Write a plugin (data write trigger)

Save as `/var/lib/influxdb3/plugins/cpu_alert.py`:

```
python

def process_writes(influxdb3_local, table_batches, args=None):
    threshold = float(args.get("threshold", "80")) if args else 80
    for batch in table_batches:
        if batch["table_name"] != "cpu":
            continue
        for row in batch["rows"]:
            usage = row.get("usage_user")
            host = row.get("host")
            if usage is not None and usage > threshold:
                influxdb3_local.warn(
                    f"High CPU on {host}: {usage:.1f}% > {threshold}%"
                )
```

15.4 Create the trigger

bash

```
influxdb3 create trigger \  
  --database metrics \  
  --trigger-spec "table:cpu" \  
  --path cpu_alert.py \  
  --trigger-arguments threshold=85 \  
  --token $INFLUXDB3_AUTH_TOKEN \  
  cpu_alert_trigger
```

15.5 Scheduled trigger example

`/var/lib/influxdb3/plugins/hourly_rollup.py`:

python

```
from datetime import datetime, timedelta  
  
def process_scheduled_call(influxdb3_local, call_time, args=None):  
    rows = influxdb3_local.query(  
        "SELECT host, mean(usage_user) AS avg_user "  
        "FROM cpu "  
        "WHERE time > now() - INTERVAL '1 hour' "  
        "GROUP BY host"  
    )  
    for row in rows:  
        line = (  
            f"cpu_hourly,host={row['host']} "  
            f"avg_user={row['avg_user']}"  
        )  
        influxdb3_local.write(line)  
    influxdb3_local.info(f"Rolled up {len(rows)} hosts")
```

bash

```
influxdb3 create trigger \  
  --database metrics \  
  --trigger-spec "every:1h" \  
  --path hourly_rollup.py \  
  --token $INFLUXDB3_AUTH_TOKEN \  
  hourly_rollup
```

15.6 HTTP request trigger example

`/var/lib/influxdb3/plugins/health_endpoint.py`:

python

```
import json

def process_request(influxdb3_local, query_parameters,
                    request_headers, request_body, args=None):
    rows = influxdb3_local.query("SELECT count(*) AS n FROM cpu")
    return {
        "status": "ok",
        "row_count": rows[0]["n"] if rows else 0
    }
```

bash

```
influxdb3 create trigger \
  --database metrics \
  --trigger-spec "request:health" \
  --path health_endpoint.py \
  --token $INFLUXDB3_AUTH_TOKEN \
  health_trigger
```

Call it:

bash

```
curl http://localhost:8181/api/v3/engine/health
```

15.7 Install Python packages

bash

```
influxdb3 install package pandas
influxdb3 install package requests
```

For air-gapped environments, pre-install packages then disable runtime installs:

bash

```
influxdb3 serve --package-manager disabled ...
```

15.8 Update a trigger's plugin code

```
bash

influxdb3 update trigger \
  --database metrics \
  --trigger-name cpu_alert_trigger \
  --path /home/shri/cpu_alert_v2.py \
  --token $INFLUXDB3_AUTH_TOKEN
```

15.9 Delete or disable a trigger

```
bash

# Disable
influxdb3 disable trigger cpu_alert_trigger \
  --database metrics --token $INFLUXDB3_AUTH_TOKEN

# Enable
influxdb3 enable trigger cpu_alert_trigger \
  --database metrics --token $INFLUXDB3_AUTH_TOKEN

# Delete
influxdb3 delete trigger cpu_alert_trigger \
  --database metrics --token $INFLUXDB3_AUTH_TOKEN
```

15.10 Multi-node placement

Plugin type	Run on
<code>table:</code> / <code>all_tables</code>	Ingest nodes
<code>every:</code> / <code>cron:</code>	Any node with <code>process</code> mode
<code>request:</code>	Nodes that serve HTTP API traffic (typically query nodes)

Make sure the plugin file exists at the same path on every node that may execute it.

16. System Tables and Monitoring

System tables live in the `_internal` database and the `system` schema of every user database.

16.1 Useful system tables

Table	Description
<code>system.databases</code>	Configured databases
<code>system.tables</code>	Tables and their schemas
<code>system.queries</code>	Recent query history
<code>system.query_log</code>	Query telemetry
<code>system.processing_engine_triggers</code>	Trigger definitions
<code>system.processing_engine_logs</code>	Plugin log output and errors
<code>system.plugin_files</code>	Loaded plugin files
<code>system.last_caches</code>	Configured LVCs
<code>system.distinct_caches</code>	Configured DVCs
<code>system.parquet_files</code>	Persisted Parquet files
<code>system.compactions</code>	Compaction history

16.2 Query examples

```
bash
```

```
# Slowest 10 queries in the last hour
influxdb3 query --database _internal --token $INFLUXDB3_AUTH_TOKEN \
  "SELECT issue_time, query_text, success, duration
  FROM system.queries
  WHERE issue_time > now() - INTERVAL '1 hour'
  ORDER BY duration DESC
  LIMIT 10"

# Recent plugin errors
influxdb3 query --database metrics --token $INFLUXDB3_AUTH_TOKEN \
  "SELECT trigger_name, log_level, log_text
  FROM system.processing_engine_logs
  WHERE log_level IN ('ERROR','WARN')
  ORDER BY event_time DESC
  LIMIT 50"

# Plugin file inventory
influxdb3 query --database _internal --token $INFLUXDB3_AUTH_TOKEN \
  "SELECT plugin_name, file_name, size_bytes
  FROM system.plugin_files
  ORDER BY plugin_name"
```

16.3 `/metrics` endpoint (Prometheus)

InfluxDB 3 exposes Prometheus-format metrics at `/metrics`:

```
bash

curl http://localhost:8181/metrics
```

Scrape with Prometheus:

```
yaml

scrape_configs:
  - job_name: influxdb3
    static_configs:
      - targets:
          - 'ingest-01.example.com:8181'
          - 'query-01.example.com:8181'
          - 'compact-01.example.com:8181'
```

16.4 `_internal` retention

The `_internal` database defaults to **7-day** retention. Only the operator admin token can change it:

```
bash

influxdb3 update database _internal \
  --retention-period 30d \
  --token $INFLUXDB3_AUTH_TOKEN
```

17. Performance Tuning

17.1 Memory tuning

Option	Default	Notes
<code>--exec-mem-pool-bytes</code>	20% of system RAM	DataFusion query memory
<code>--force-snapshot-mem-threshold</code>	50% of system RAM	Triggers WAL snapshot
<code>--parquet-mem-cache-size</code>	20% of system RAM	Parquet file cache

Override in absolute bytes or percentages:

```
bash

--parquet-mem-cache-size 4GB
--exec-mem-pool-bytes 30%
```

17.2 Thread allocation

```
bash

--num-cores 8          # cap total worker threads
```

17.3 WAL replay concurrency

Defaults to the number of CPU cores; tune down to limit memory spikes during startup:

```
bash
```

```
--wal-replay-concurrency-limit 4
```

17.4 Snapshot checkpointing (faster startup)

Periodically consolidate snapshots into monthly checkpoints to speed up server boot:

```
bash
--checkpoint-interval 1h
```

17.5 Catalog limits (Enterprise)

```
bash
--num-database-limit 1000
--num-table-limit 10000
--num-total-columns-per-table-limit 1000
```

17.6 Compactor placement

Run compaction on dedicated nodes (`--mode compact`) so ingest and query workloads are not affected by background compaction CPU and memory usage.

17.7 Query node tuning

For query-heavy workloads:

- Increase `--parquet-mem-cache-size` to 40-50% of RAM.
- Enable LVC and DVC on hot tables.
- Add file indexes on filter columns.
- Use SSD-backed `--data-dir` for the local Parquet cache.

18. Performance Upgrade Preview (PachaTree)

Beta only — do not use in production. This feature is intended for staging or test environments.

The performance upgrade preview introduces a new columnar file format (`.pt` files), column families, and bounded compaction.

18.1 Enable the preview

```
bash

influxdb3 serve \
  --node-id host01 \
  --cluster-id cluster01 \
  --object-store file \
  --data-dir /var/lib/influxdb3/data \
  --use-pacha-tree
```

Or with an environment variable:

```
bash

export INFLUXDB3_ENTERPRISE_USE_PACHA_TREE=true
influxdb3 serve ...
```

18.2 Column families

Use `::` (double colon) in field names to group fields:

```
metrics,host=srv01
cpu::usage_user=55.2,cpu::usage_sys=12.1,mem::free=2048i,mem::used=6144i
```

This creates two families: `cpu` (2 fields) and `mem` (2 fields). Queries that touch only `mem::free` skip the `cpu` family on disk.

18.3 Hybrid query mode

When enabled on a cluster with existing Parquet data, the server enters hybrid mode and:

1. Detects existing Parquet files.
2. Streams them through a conversion pipeline to `.pt` format.
3. Queries merge results from both formats during migration.

Monitor progress:

```
sql

SELECT * FROM system.upgrade_parquet_node;
SELECT * FROM system.upgrade_parquet;
```

18.4 Export to Parquet

```
bash
```

```
influxdb3 export databases
influxdb3 export tables -d metrics
influxdb3 export windows -d metrics -t cpu
influxdb3 export data -d metrics -t cpu -o ./export_output
```

18.5 Downgrade back to Parquet

```
bash
```

```
# Stop all nodes first
sudo systemctl stop influxdb3-enterprise
```

```
# Dry run
```

```
influxdb3 downgrade-to-parquet \
  --cluster-id datapundit-cluster \
  --object-store file \
  --data-dir /var/lib/influxdb3/data \
  --dry-run
```

```
# Execute
```

```
influxdb3 downgrade-to-parquet \
  --cluster-id datapundit-cluster \
  --object-store file \
  --data-dir /var/lib/influxdb3/data
```

Warning: Downgrade deletes `.pt` files. Only original Parquet data is preserved.

19. Security Hardening

19.1 Enable TLS

```
bash
```

```
--tls-cert /etc/influxdb3/tls/server.crt \
--tls-key /etc/influxdb3/tls/server.key \
--tls-minimum-version tls-1.3
```

For internal CAs:

```
bash
```

```
export INFLUXDB3_TLS_CA=/etc/ssl/certs/internal-ca.pem
```

19.2 systemd sandboxing

DEB/RPM installs run InfluxDB under a systemd unit with these protections enabled by default:

- `PrivateTmp=yes`
- `ProtectSystem=strict`
- `ProtectHome=yes`
- `NoNewPrivileges=yes`
- Dedicated `influxdb3` user, no shell access

To audit:

```
bash
systemctl cat influxdb3-enterprise
```

19.3 Bind to a specific interface

```
bash
--http-bind 10.0.1.5:8181
```

19.4 Disable unauthorized endpoints

By default, `/health`, `/ping`, and `/metrics` can be opted out of authentication. To require auth on them:

```
bash
--require-auth-for /health,/ping,/metrics
```

19.5 Plugin path validation

The Processing Engine blocks:

- Parent directory traversal (`../`)
- Absolute paths outside `--plugin-dir`
- Symlinks that escape the plugin directory

Plugin upload and update operations require an **admin token**.

19.6 Disable plugin uploads (production)

Pre-deploy plugins via secure file transfer and lock the engine down:

```
bash
--package-manager disabled
```

19.7 CORS

Browser clients can issue cross-origin requests. To restrict origins, configure your reverse proxy (NGINX/HAProxy) to filter `Origin` headers and CORS responses.

19.8 Recommended firewall rules

Port	Direction	Source	Purpose
8181	Inbound	App / load balancer	HTTP API
22	Inbound	Admin bastion only	SSH
9090	Inbound	Prometheus	Metrics scrape
443/9000	Outbound	Object store	S3/MinIO/Azure/GCS

20. Backup and Restore

InfluxDB 3 Enterprise stores all durable state in the **object store** (or `--data-dir` when using local filesystem). Backups are taken at the object store layer.

20.1 What to back up

Path	Purpose
<code><bucket>/catalog/</code>	Catalog (databases, tables, tokens, plugins, schemas)
<code><bucket>/dbs/</code>	Persisted Parquet data
<code><bucket>/wal/</code>	Write-Ahead Log segments
<code><bucket>/snapshots/</code>	Snapshot checkpoints

Path	Purpose
Local <code>--plugin-dir</code>	Plugin source files
<code>/etc/influxdb3/influxdb3-enterprise.conf</code>	Server config
License file under <code>--data-dir</code>	License

20.2 Backup order (multi-node)

To get a consistent point-in-time backup:

1. **Quiesce ingest** — pause writes via your load balancer (or accept eventually-consistent backups).
2. **Snapshot the object store** — use the cloud provider snapshot tool, `rclone copy`, or `aws s3 sync`.
3. **Copy plugin directory** — `rsync -a /var/lib/influxdb3/plugins/ backup-host:/backup/plugins/`.
4. **Copy config and license** — `tar -czf influxdb3-config.tgz /etc/influxdb3/`.

20.3 Example: S3 backup with `aws s3 sync`

```
bash

aws s3 sync \
  s3://influxdb3-data \
  s3://influxdb3-backup-$(date +%Y%m%d) \
  --storage-class GLACIER_IR
```

20.4 Example: Local-disk backup with rsync

```
bash

sudo systemctl stop influxdb3-enterprise

rsync -av --delete \
  /var/lib/influxdb3/ \
  /backup/influxdb3/$(date +%Y%m%d)/

sudo systemctl start influxdb3-enterprise
```

20.5 Restore

1. Stop all InfluxDB 3 Enterprise nodes.
2. Restore the object-store contents to the **original bucket and path** (or update `--bucket` to point at the restore location).
3. Restore plugin files to `--plugin-dir` on every node that runs plugins.
4. Restore the license file.
5. Start nodes in this order: **compact** → **ingest** → **query**.
6. Verify with `influxdb3 show databases` and a sample query.

20.6 Disaster recovery considerations

- Use cross-region replication for the object store (S3 CRR, GCS multi-region, etc.).
 - Keep an offline copy of the license file.
 - Document and test the restore procedure quarterly.
-

21. Upgrading InfluxDB 3 Enterprise

21.1 Read the release notes

Always read the release notes for **every intermediate version** before upgrading. Pay attention to:

- Catalog version changes (may block downgrade).
- Configuration option renames or removals.
- Breaking API changes.

21.2 Pre-upgrade checklist

- ☐ Take a backup (section 20).
- ☐ Confirm license is still valid.
- ☐ Verify node-by-node disk space for new binaries.
- ☐ Verify version compatibility across the cluster (do not mix major versions for long).

21.3 Rolling upgrade (multi-node)

Upgrade nodes in this order to minimize downtime:

1. **Compact nodes** — least user-facing impact.
2. **Query nodes** — drain via load balancer, upgrade, return to pool.

3. Ingest nodes — drain via load balancer, upgrade, return to pool.

Per-node procedure (DEB/RPM):

```
bash

# Drain from LB

sudo systemctl stop influxdb3-enterprise
sudo apt-get update && sudo apt-get install --only-upgrade influxdb3-enterprise #
# or
sudo yum upgrade influxdb3-enterprise #

sudo systemctl start influxdb3-enterprise
sudo journalctl -u influxdb3-enterprise -f

# Health check before returning to LB
curl -s http://localhost:8181/health
```

21.4 Single-node upgrade

```
bash

sudo systemctl stop influxdb3-enterprise
sudo yum upgrade influxdb3-enterprise
sudo systemctl start influxdb3-enterprise
influxdb3 --version
```

21.5 Identify your installed version

```
bash

influxdb3 --version
curl -s http://localhost:8181/ping | jq .
```

22. Troubleshooting

22.1 Server fails to start — license required

```
Error: a valid license is required to start
```

Run `influxdb3 serve --license-type trial --license-email you@example.com ...` or follow the interactive onboarding URL printed at startup.

22.2 `influxdb3` cannot find the `python/` directory

If using a binary install:

```
Error: failed to initialize embedded Python runtime
```

Make sure `influxdb3` lives in the same parent directory as the `python/` folder, and `PATH` points at that parent — not a copy of the binary moved elsewhere.

22.3 Writes rejected with "field exceeds 1MB"

Default v3.9 string field limit is 1 MB. To raise it:

```
bash

--max-string-field-size 4MB
```

22.4 `unknown database` after a restart

Soft-deleted databases are reported as missing. Check:

```
bash

influxdb3 query --database _internal --token $INFLUXDB3_AUTH_TOKEN \
  "SELECT name, deleted, hard_deletion_date FROM system.databases"
```

22.5 Trigger errors

Check the trigger log table:

```
sql

SELECT event_time, log_level, log_text
FROM system.processing_engine_logs
WHERE trigger_name = 'cpu_alert_trigger'
ORDER BY event_time DESC
LIMIT 100;
```

If errors are persistent, set `--error-behavior disable` so the trigger auto-disables instead of looping.

22.6 Object store connectivity issues

```
bash

# Test S3 reachability from the host
aws s3 ls s3://influxdb3-data/

# Test MinIO
curl -I http://minio.internal:9000/minio/health/ready
```

Check the InfluxDB log for `object_store` errors and confirm credentials, region, and endpoint are correct.

22.7 High WAL replay times on startup

- Lower `--wal-replay-concurrency-limit` if you are running out of memory.
- Enable `--checkpoint-interval 1h` so future restarts have fewer snapshots to replay.

22.8 Useful log filter levels

```
bash

--log-filter info      # default
--log-filter debug     # verbose, for triage
--log-filter trace     # extremely verbose, short bursts only
```

Set per-module:

```
bash

--log-filter "info,influxdb3_write=debug,object_store=debug"
```

Appendix A — Quick Reference Cheat Sheet

```
bash
```

```
# --- Service ---
sudo systemctl start|stop|restart|status influxdb3-enterprise
sudo journalctl -u influxdb3-enterprise -f

# --- Tokens ---
influxdb3 create token --admin
influxdb3 show tokens --token $T
influxdb3 delete token --name foo --token $T

# --- Databases ---
influxdb3 create database metrics --token $T
influxdb3 show databases --token $T
influxdb3 update database metrics --retention-period 30d --token $T
influxdb3 delete database metrics --token $T

# --- Tables ---
influxdb3 create table cpu --database metrics --tags host,region \
    --fields usage_user:float64 --token $T
influxdb3 show retention --database metrics --token $T

# --- Write ---
influxdb3 write --database metrics --token $T \
    'cpu,host=srv01 usage_user=55.2'

# --- Query ---
influxdb3 query --database metrics --token $T \
    "SELECT * FROM cpu LIMIT 10"

# --- Caches ---
influxdb3 create last_cache --database metrics --table cpu --name lvc1 \
    --key-columns host --value-columns usage_user --count 5 --token $T
influxdb3 create distinct_cache --database metrics --table cpu --name dvc1 \
    --columns host --token $T

# --- Triggers ---
influxdb3 create trigger --database metrics \
    --trigger-spec "every:1m" --path script.py --token $T my_trigger
influxdb3 show triggers --database metrics --token $T

# --- License ---
influxdb3 show license --token $T
```

Appendix B — Suggested Lab Exercises (datapundit training tie-in)

Module	Exercise
1	Install on Rocky Linux from RPM; verify health and version.
2	Activate trial license; create operator admin token.
3	Create a database, write 10k points via CLI, query with SQL.
4	Configure Telegraf system input → InfluxDB 3 write.
5	Build a 3-node cluster (ingest/query/compact) on MinIO.
6	Add LVC + DVC + file index; benchmark before/after.
7	Write a Python plugin: scheduled hourly rollup.
8	Write an HTTP request plugin: custom <code>/health-deep</code> endpoint.
9	Backup and restore from S3 with <code>aws s3 sync</code> .
10	Rolling upgrade 3.8.x → 3.9.x with zero ingest downtime.

End of self-hosted deployment guide.